

第8章 代数与数值算法

8.1 引言

- 算法设计考虑因素：
 - 复杂性（算法的时间效率）
 - 收敛性（防止数据溢出）
 - 可靠性（误差在运算过程中会累积，需要评估误差比例）
- 误差来源：
 - 模型误差
 - 观测误差
 - 截断误差
 - 舍入误差

8.2 求幂运算

- 问题：将单步乘法视为 $O(1)$ 操作，设计高效算法求 n^k
- 直接归纳： $n^k = n \cdot n^{k-1}$
时间复杂度 $O(k) = O(2^{\log k})$ 为指数时间算法
- 分治归纳： $n^k = (k \% 2 == 0) ? (n^{k/2})^2 : n \cdot (n^{k/2})^2$
时间复杂度 $O(\log k)$ 为线性时间算法

8.3 最大公约数

- 问题：求 n, m 的最大公约数
- 直接归纳法：从 2 开始尝试，直到 n 或 m ，时间复杂度 $O(n + m)$
- 辗转相除法：不妨设 $n > m$ ，设 $GCD(n, m)$ 表示 n, m 的最大公约数
 $n \% m \neq 0$ 时有 $GCD(n, m) = GCD(n \% m, m)$
 $n \% m = 0$ 时 n, m 的最大公约数就是 m
时间复杂度 $O(\log(n + m))$

8.4 多项式乘积

- 问题：求 $P = p_0 + p_1x + p_2x^2 + \dots + p_nx^n$ 与 $Q = q_0 + q_1x + q_2x^2 + \dots + q_nx^n$ 的乘积多项式
- 直接法：时间复杂度 $O(n^2)$

• **分治法:**

$$P_1 = p_0 + p_1x + p_2x^2 + \cdots + p_{\frac{n}{2}-1}x^{\frac{n}{2}-1}$$

$$P_2 = p_{\frac{n}{2}} + p_{\frac{n}{2}+1}x + p_{\frac{n}{2}+2}x^2 + \cdots + p_nx^{\frac{n}{2}}$$

$$Q_1 = q_0 + q_1x + q_2x^2 + \cdots + q_{\frac{n}{2}-1}x^{\frac{n}{2}-1}$$

$$Q_2 = q_{\frac{n}{2}} + q_{\frac{n}{2}+1}x + q_{\frac{n}{2}+2}x^2 + \cdots + q_nx^{\frac{n}{2}}$$

$$PQ = (P_1 + P_2x^{\frac{n}{2}})(Q_1 + Q_2x^{\frac{n}{2}}) = P_1Q_1 + (P_1Q_2 + P_2Q_1)x^{\frac{n}{2}} + P_2Q_2x^n$$

$$\text{时间复杂度 } T(n) = 4T(\frac{n}{2}) + O(n) \quad T(n) = O(n^2)$$

• **分治优化:** 只需要知道 $P_1Q_2 + P_2Q_1$, 无需分别求出 P_1Q_2, P_2Q_1

$$P_1Q_2 + P_2Q_1 = (P_1 + P_2)(Q_1 + Q_2) - P_1P_2 - Q_1Q_2$$

$$\text{时间复杂度 } T(n) = 3T(\frac{n}{2}) + O(n) \quad T(n) = O(n^{\log_2 3}) \approx O(n^{1.59})$$

• **应用:** 在计算大整数乘法时, 设每个整数有 $O(n)$ 位, 若将 1 位加法视为 $O(1)$ 操作, 可用类似的分治优化思路, 将时间复杂度从 $O(n^2)$ 降为 $O(n^{\log_2 3}) \approx O(n^{1.59})$

8.5 方阵乘法

• **定义:** $C_n = A_n B_n \Leftrightarrow c_{ij} = \sum_{k=1}^n a_{ik} b_{kj}$ 时间复杂度 $O(n^3)$, 其中 n 为方阵的阶数

• **分治:**

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \quad B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}$$

$$C = AB = \begin{pmatrix} A_{11}B_{11} + A_{12}B_{21} & A_{11}B_{12} + A_{12}B_{22} \\ A_{21}B_{11} + A_{22}B_{21} & A_{21}B_{12} + A_{22}B_{22} \end{pmatrix}$$

$$T(n) = 8T\left(\frac{n}{2}\right) + O(n^2) \quad T(n) = O(n^3)$$

• **优化:** 类似于多项式乘法优化, 经过组合后只需要算 7 个乘法

$$T(n) = 7T\left(\frac{n}{2}\right) + O(n^2) \quad T(n) = O(n^{\log_2 7}) \approx O(n^{2.81})$$

缺点: n 非常大时才有优化, 输出误差较大, 不易并行化

8.6 布尔阵乘法

• **问题:** 矩阵的每个元素都是布尔值, 以逻辑与替换乘法, 以逻辑或替换加法
与数值矩阵运算的不同之处: 不能计算减法

• **寻址法:** 做乘法相加的步骤可视为向量内积

将向量分段, 每一段长为 k , 可在 $O(k \cdot 2^k)$ 时间内算出所有可能的 k 维向量内积值, 之后计算向量内积时, 可以按地址查找结果

$k = \lfloor \frac{1}{2} \log_2 n \rfloor$ 时, 构造表时间为 $O(n \log n)$, 可在 $O(\frac{n}{k}) = O(n \log n)$ 时间内计算 k 维向量的内积, 计算 n^2 次的复杂度为 $O(\frac{n^3}{\log n})$, 总时间复杂度 $O(\frac{n^3}{\log n} + n \log n)$

k 增大时, 空间复杂度增加, 时间复杂度减小

• **分块法:** AB 可视为 A 第 i 列的每个元素与 B 第 i 行的每个元素相乘, 得到矩阵 M_i , 即

$$M_i[kl] = A[ki]B[il], \text{ 则 } C = \sum_i M_i$$

为加速操作, 可一次性取 A 的 k 列与 B 的 k 行相乘, 具体实现为根据 A 第 i 行的元素, 取出

B 的某些行相加，得到结果矩阵的一行

相加操作可用预计算查表的方式实现，而由于 A 的每行均已确定，可能的组合只有 2^k 种，而查找到结果后需要以 $\Theta(n)$ 时间复制到结果矩阵，所以根据表计算 M_i 的时间复杂度为 $\Theta(n^2)$

构建表的方法是：记 A 的 k 行可能的组合为 $(0, 1)_k$ ，则全为 0 时结果为 0，否则找到最低位 1 的位置，求 $xxx\dots x1000\dots 0$ 对应值即为 $xxx\dots x0000\dots 0$ 对应值与 $000\dots 01000\dots 0$ 对应值之和，所以表中每行计算时间均为 $\Theta(n)$ ，构建表的复杂度为 $O(n \cdot 2^k)$

总时间复杂度 $\frac{n}{k} [O(n^2) + O(n \cdot 2^k)]$ ，当 $k = \log n$ 时复杂度为 $O(\frac{n^3}{\log n})$

优化：布尔加法也可用预计算查表的方式实现，可将复杂度降为 $O(\frac{n^3}{\log^2 n})$

8.7 矩阵连乘

- **引入**：根据定义计算 $M_{ij}M_{jk}$ 的时间为 ijk

对于 $M_{ij}M_{jk}M_{kl}$ ，不同计算方式的计算次数不同

- $(M_{ij}M_{jk})M_{kl}$ 的时间为 $ijk + ikm$

- $M_{ij}(M_{jk}M_{kl})$ 的时间为 $jkm + ijm$

- **问题**：求 n 个矩阵（不一定是方阵）连乘最少计算次数的计算顺序

- **暴力遍历法**：令 $M'_1 = \prod_{i=1}^k M_i, M'_2 = \prod_{i=k+1}^n M_i$ ，则 $\prod_{i=1}^n M_i = M'_1 M'_2$ ，计算方法数 $f(n)$ 满足 $f(n) = \sum_{k=1}^{n-1} f(k)f(n-k)$

$f(n)$ 为卡特兰数 C_{n-1} ，因此有 $f(n) = \frac{1}{n} \binom{2n-2}{n-1} = \frac{1}{n} \cdot \frac{(2n-2)!}{((n-1)!)^2} = O(\frac{4^n}{n^{1.5}})$ ，计算每种方法的计算次数的复杂度 $\Theta(n)$ ，所以总复杂度 $O(\frac{4^n}{\sqrt{n}})$ ，仅比指数时间稍低

- **动态规划法**： $C[i, j] = \min_{1 \leq k \leq j} \{C[i, k-1] + C[k, j] + r_i r_k r_{j+1}\}$ ，问题的解为 $C[1, n]$
复杂度分析：计算动态规划表每个元素的复杂度 $O(n)$ ，表的规模为 $\Theta(n^2)$ ，总时间复杂度 $O(n^3)$

8.8 快速傅里叶变换

- **思想**：多项式的函数曲线及解析式可以用多个点确定

- **方法**： $r(x) = p(x)q(x)$ ，设 $p(x)$ 为 m 次多项式， $q(x)$ 为 n 次多项式，则选取 $m+n+1$ 个不同点 $x_1, x_2, x_3, \dots, x_{m+n+1}$ 计算 $p(x_i)q(x_i)$ ，然后解方程组求解 $r(x)$ ，时间复杂度 $O((m+n)^2)$

- **优化**：选择特殊点降低计算量

例如，选择 x_i, x_j 时令 $x_i = -x_j$ ，可减少一半的计算量，即分别求出奇次项和偶次项的绝对值，然后根据符号求和，就能在 $O(n)$ 时间内求出 $P(x_i), P(-x_i)$

时间复杂度可优化为 $T(n) = 2T(\frac{n}{2}) + O(n)$ ，但只对 n 成立，即只能分治 1 次

若要多次递归分治，则需要引入复数域、四重数域……，可使上述递归复杂度恒成立，因此最终的复杂度 $T(n) = O(n \log n)$

解方程组求解 $r(x)$ ：

$$\begin{bmatrix} 1 & 1 & 1 & \cdots & 1 \\ 1 & \omega & \omega^2 & \cdots & \omega^{n-1} \\ 1 & \omega^2 & \omega^4 & \cdots & \omega^{2(n-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{n-1} & \omega^{2(n-1)} & \cdots & \omega^{(n-1)(n-1)} \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ \vdots \\ a_{n-1} \end{bmatrix} = \begin{bmatrix} v_0 \\ v_1 \\ v_2 \\ \vdots \\ v_{n-1} \end{bmatrix}$$

$$M(\omega)a = v \Rightarrow a = M(\omega)^{-1}v$$

$$\therefore M(\omega)^{-1} = \frac{1}{n}M\left(\frac{1}{\omega}\right) \quad \therefore a = \frac{1}{n}M\left(\frac{1}{\omega}\right)v$$

8.9(*) 非线性方程组数值解

- 引入：若将重根视为多个解，则一元 n 次方程在复数域总有 n 个解
- 二分法：逐步缩小区间，循环结束条件： $\Delta x < \epsilon$ 或 $|f(x)| < \epsilon$
- 迭代法：令 $g(x) = f(x) + x$ ，则 $f(x) = 0 \Leftrightarrow g(x) = f(x) + x$ ，当 $g(x)$ 收敛于其不动点时可用 $x_{n+1} = g(x_n)$ 的方式简单迭代求解
- 牛顿切线法： $x_{k+1} \approx x_k - \frac{f'(x_k)}{f''(x_k)}$

局部收敛性：若 $f''(x)$ 连续， $f'(x) \neq 0$ ，则存在根的邻域，使得牛顿切线法的收敛速度不低于 2 阶，即

$$\lim_{k \rightarrow \infty} \frac{x_{k+1} - x_0}{(x_k - x_0)^2} = 0$$